



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Fecha:

DD/MM/2026

Organización de lenguajes y compiladores 2

Escuela de Ingeniería de Ciencias Y Sistemas

**NOMBRE DEL
TUTOR**



Traducción de estructuras de control en un compilador

Agenda

- Entorno de Ejecución
-

Recordatorios:

- Foro de esta semana
- Calificación Tarea 3
- Hoja de calificación



Recursos:

- [ARM64 Cheat Sheet](#)
- [StackScope](#)

Valor:

Responsabilidad Técnica

Optimización de Código — Optimización de Bucles

La optimización de bucles consiste en aplicar transformaciones al cuerpo o estructura de un bucle con el objetivo de mejorar el rendimiento durante la ejecución, sin alterar el resultado final del programa. Importancia:

- Los bucles representan las secciones de código más ejecutadas en la mayoría de los programas; por tanto, optimizarlos puede generar mejoras significativas en el tiempo total de ejecución.

Principales técnicas

- Desenrollado del bucle (Loop Unrolling): reduce la cantidad de iteraciones duplicando el cuerpo del bucle para minimizar el número de saltos y comprobaciones de condición.
- Invarianza de código (Loop-Invariant Code Motion): mueve fuera del bucle las operaciones cuyos resultados no dependen de las iteraciones.
- Fusión de bucles (Loop Fusion): combina varios bucles consecutivos que recorren el mismo rango para reducir el overhead de control.
- División de bucles (Loop Splitting): separa bucles complejos en varios más simples para mejorar la paralelización y la localización de datos.

Optimización de Código — Simplificación Algebraica

La simplificación algebraica es una técnica de optimización que aplica reglas matemáticas para reemplazar expresiones por versiones equivalentes más simples o eficientes.

Objetivo:

- Reducir el número de operaciones aritméticas, eliminar redundancias y mejorar la legibilidad del código intermedio generado por el compilador.

Ejemplos de transformaciones comunes:

- Reemplazar multiplicaciones o divisiones por potencias de dos con desplazamientos de bits.
- Sustituir expresiones como $x + 0$, $x * 1$, $x * 0$, o $x - 0$ por sus equivalentes simplificados.
- Aplicar propiedades distributivas o asociativas para reducir cálculos repetitivos.

Optimización de Código — Optimización de Saltos Condicionales

La optimización de saltos condicionales tiene como objetivo reducir la cantidad de bifurcaciones innecesarias y simplificar el flujo de control generado por el compilador.

Motivación:

- Los saltos condicionales excesivos degradan el rendimiento, ya que provocan interrupciones en la ejecución secuencial y afectan la predicción de flujo en los procesadores modernos.

Principales técnicas

- Simplificación de condiciones: reemplazar expresiones lógicas redundantes o triviales (por ejemplo, convertir `if (true)` en ejecución directa).
- Eliminación de saltos innecesarios: eliminar secuencias donde un salto condicional va seguido de un salto incondicional al mismo destino.
- Combinación de condiciones: fusionar múltiples bifurcaciones consecutivas en una única evaluación lógica.
- Predicación: transformar saltos simples en instrucciones condicionales directas, evitando bifurcaciones explícitas.

Recolección de Basura — Garbage Collection

El Garbage Collection (GC) es un proceso automático mediante el cual el sistema de ejecución o el compilador identifica y libera la memoria ocupada por objetos que ya no son accesibles en el programa.

Objetivo:

- Evitar fugas de memoria y mantener una administración eficiente de los recursos durante la ejecución, sin requerir intervención explícita del programador.

Principales estrategias

- Conteo de referencias: cada objeto mantiene un contador que indica cuántas referencias apuntan a él; cuando llega a cero, se libera la memoria.
- Barrido y marcado (Mark and Sweep): el recolector marca los objetos accesibles desde las raíces y elimina los no marcados.
- Copia y compactación: los objetos vivos se copian a un nuevo espacio de memoria, agrupándolos y liberando el espacio restante.
- Recolección generacional: divide los objetos por antigüedad, aplicando recolecciones más frecuentes a los de vida corta.

Recolección de Basura — Memory Leaks

Un memory leak (fuga de memoria) ocurre cuando un programa reserva memoria dinámica pero no la libera, incluso cuando ya no es accesible o necesaria.

Causas comunes:

- Pérdida de referencias a objetos asignados en el heap.
- Estructuras de datos con referencias circulares sin mecanismos de liberación.
- Errores en la gestión manual de memoria en lenguajes sin recolección automática.

Consecuencias:

- Incremento progresivo del consumo de memoria.
- Degradación del rendimiento del sistema.
- Fallos por agotamiento de recursos.

Relación con el compilador y el GC

- En lenguajes con garbage collection, los leaks se minimizan, pero aún pueden ocurrir si las referencias permanecen activas innecesariamente.
- En lenguajes de administración manual, el compilador puede ofrecer advertencias o herramientas de análisis estático para detectar posibles fugas.

The screenshot displays the Visual Studio Code interface with the following components:

- Editor:** Shows assembly code for a program. The code includes a `.global _start` directive, a `.section .text` directive, and a `_start` function. The function calls `exit(0)` and then enters a `main` function. The `main` function calls `sum(1, 2)`, which returns the value 3 in register `x0`. The program then converts the result to a string and prints it.
- Debug Console:** Shows the execution flow, including the `main` function and the `sum` function. The console output shows the program's output: `3`.
- Registers:** A table showing the current state of registers. The registers are `$sp`, `$lr`, `x29`, `x0`, `x1`, and `x8`. The values are `0x55007ff760`, `0x4000b4`, `0x0`, `0x0`, `0x0`, and `0x0` respectively.

The status bar at the bottom indicates the program is stopped, the session is `60dad951...`, the address is `$sp`, and the architecture is `ARM64`.

Ejemplo: Debugging en VSCode



Implementación